

Smartshift

Vollstaendige Erklarung des Quellcodes — Schichtplanungssoftware (FastAPI + SQLite + HTML/JS)

Architektur-Ueberblick

Das Backend ist dreischichtig aufgebaut. Der Browser kommuniziert per HTTP/JSON mit FastAPI, das ueber die CRUD-Schicht auf die SQLite-Datenbank zugreift. Zwei Spezialmodule (Scheduler, Gemini-Assistent) erweitern die Logik.

```
Browser (index.html + app.js + styles.css)
  ↓ HTTP/JSON (fetch)
FastAPI (main.py) → crud.py → SQLAlchemy (models.py) → SQLite
  → scheduler.py (Planungsalgorithmus)
  → gemini_assistant.py (optionale KI)
```

- **main.py** = API-Schicht (nimmt HTTP-Anfragen entgegen)
- **crud.py** = Logik-Schicht (Datenbankoperationen)
- **models.py** = Datenschicht (Tabellen-Definitionen)
- **schemas.py** = Validierung der Ein- und Ausgaben
- **scheduler.py / gemini_assistant.py** = Spezialmodule

1. database.py – Datenbankverbindung

Die kleinste Datei. Stellt die Verbindung zur SQLite-Datei her.

Element	Funktion
DB_PATH / DATABASE_URL	Berechnet den Pfad zu data/smartshift.db
engine	SQLAlchemy-Verbindung. check_same_thread=False, da FastAPI mehrere Threads nutzt
SessionLocal	Fabrik, die einzelne Datenbank-Sitzungen erzeugt
init_db()	Erstellt den data-Ordner und legt alle Tabellen an. Laeuft beim Start
get_db()	Liefert pro Anfrage eine Session und schliesst sie sicher (yield + finally)

2. models.py – Die Datenbank-Tabellen

Jede Klasse entspricht einer Tabelle. Zuerst drei Enums (feste Auswahllisten): **IndustryType** (restaurant/retail/factory), **ExperienceLevel** (junior/normal/senior), **AbsenceType** (vacation/sick/school/other).

Klasse	Tabelle	Speichert
Business	businesses	Betrieb: Name, Branche
OpeningHour	opening_hours	Oeffnungszeit pro Wochentag (0=Mo)
Area	areas	Bereich (z.B. Kueche)

Klasse	Tabelle	Speichert
Role	roles	Rolle (z.B. Koch)
Employee	employees	Mitarbeiter: Zielstunden, Maxstunden, Level, freie Tage
EmployeeRole	employee_roles	Verknuepfung: welcher MA hat welche Rolle
EmployeeArea	employee_areas	Verknuepfung: welcher MA darf in welchem Bereich
Availability	availability	Verfuegbarkeit an einem Datum (von/bis)
Absence	absences	Abwesenheit (Typ, von/bis, genehmigt)
ShiftRequirement	shift_requirements	Anforderung: Bereich, Rolle, Zeit, Anzahl
GeneratedShift	generated_shifts	Ergebnis: konkrete Schicht mit Soll/Ist + Grund
ShiftAssignment	shift_assignments	Verknuepfung: welcher MA in welcher Schicht

Warum Verknuepfungstabellen? Ein Mitarbeiter kann mehrere Rollen haben und eine Rolle viele Mitarbeiter (n:m-Beziehung). Geloest ueber eine Zwischentabelle. **cascade="all, delete-orphan"** loescht zugehoerige Daten automatisch mit, wenn der Betrieb geloescht wird.

3. schemas.py – Validierung (Pydantic)

Waehrend models.py definiert, WIE Daten gespeichert werden, definiert schemas.py, WIE Daten ueber die API rein- und rauskommen. Pro Entitaet meist drei Varianten:

- **...Create** – Felder, die beim Anlegen erlaubt sind
- **...Update** – Felder zum Aendern (alle optional)
- **...Out** – Felder, die zurueckgegeben werden (mit id)

Eingebaute Validierung – Beispiele:

- name: min_length=1, max_length=200 → darf nicht leer sein
- weekday: ge=0, le=6 → nur Wochentage 0–6
- target_hours_per_week: ge=0, le=168 → gueltige Stundenzahl

Zwei eigene Validatoren: Enddatum ≥ Startdatum (Abwesenheit) und Endzeit nach Startzeit (Anforderung).

4. crud.py – Datenbankoperationen

CRUD = Create, Read, Update, Delete. Jede Funktion erhaelt eine db-Session und erledigt genau eine Aufgabe. Besondere Funktionen:

- **create_business**: legt den Betrieb an UND automatisch die Standard-Bereiche/Rollen je nach Branche (Restaurant → Service/Kueche/Bar/Kasse).
- **create_employee**: legt MA an und fuellt die Verknuepfungstabellen (Rollen/Bereiche).
- **update_employee**: alte Rollen/Bereiche werden geloescht und neu gesetzt.
- **upsert_availability**: update-or-insert – verhindert doppelte Eintraege pro MA und Tag.
- **get_generated_shifts**: laedt Schichten + Bereiche + Rollen + Zuweisungen + MA in einer Abfrage (joinedload, vermeidet N+1-Problem).
- **delete_generated_shifts**: loescht alle Schichten einer Woche – wird vor jeder Neuplanung aufgerufen.

5. scheduler.py – Der Planungsalgorithmus (Herzstueck)

Keine KI – eine nachvollziehbare Heuristik. EXPERIENCE_PRIORITY: senior=0, normal=1, junior=2 (kleiner = hoehere Prioritaet).

Hilfsfunktionen

Funktion	Aufgabe
<code>_time_to_hours(t)</code>	Wandelt 17:30 in 17.5 um (zum Rechnen)
<code>_shift_duration(s, e)</code>	Dauer einer Schicht in Stunden
<code>_times_overlap(...)</code>	Ueberschneiden sich zwei Zeitfenster? (Zeitkonflikt)
<code>_is_absent(emp, datum)</code>	Ist der MA abwesend? Gibt (True, 'Krank') zurueck
<code>_is_available_for_shift</code>	Passt die eingetragene Verfuegbarkeit zur Schicht? Ohne Eintrag gilt der MA als verfuegbar
<code>_employee_role_ids / _area_ids</code>	Listet Rollen-/Bereichs-IDs eines MA

Hauptfunktion: `generate_schedule(db, business_id, week_start)`

- 1. Pruefung: week_start muss ein Montag sein.
- 2. Alten Plan dieser Woche loeschen.
- 3. Daten laden: aktive Mitarbeiter (mit Rollen, Bereichen, Abwesenheiten, Verfuegbarkeiten) und aktive Anforderungen.
- 4. Tracking anlegen: weekly_hours (Stunden je MA) und daily_assignments (belegte Zeiten je MA).
- 5. Anforderungen auf konkrete Daten umrechnen (weekday=0 → Montag).
- 6. Sortieren nach Datum, dann Startzeit.

7. Fuer jede Anforderung alle Mitarbeiter pruefen (wer durchfaellt, wird mit Grund gezaehlt):

- Passende Rolle? → sonst 'Falsche Rolle'
- Passender Bereich? → sonst 'Falscher Bereich'
- Abwesend? → 'Krank' / 'Urlaub'
- Verfuegbar zur Schichtzeit? → sonst 'Nur 9–16 Uhr verfuegbar'
- Zeitkonflikt mit anderer Schicht? → 'Zeitkonflikt'
- Wurde Maximalstunden sprengen? → 'Max. Wochenstunden erreicht'

Die uebrigen Kandidaten werden sortiert nach (1) wenigste bisherige Stunden → gleichmaessige Verteilung, dann (2) Erfahrungslevel → Senior zuerst. Die ersten N werden zugeteilt. Bei Unterbesetzung wird ein Begruendungstext erzeugt, z.B.: 'Fehlend: 1 Person(en). Gruende: 2x Krank; 1x Zeitkonflikt'. Schicht und Zuweisungen werden gespeichert.

8. Commit und Rueckgabe: {shifts_created, assignments_made, understaffed_count, warnings}. Das Besondere: Jede Ablehnung wird gezaehlt und als Klartext begruetet – volle Nachvollziehbarkeit.

6. gemini_assistant.py – Optionale KI-Analyse

Berechnet NICHTS am Plan – fasst ihn nur zusammen und laesst ihn von Gemini kommentieren.

Element	Funktion
try: import google.generativeai	Fehlt das Paket, wird GENAI_AVAILABLE=False gesetzt statt abzustuerzen
_build_prompt(summary)	Baut einen deutschen Prompt: Wochenuebersicht, Schichten, MA-Stunden + 4 Fragen
analyze_schedule(summary)	Dreifache Absicherung: kein Key / Paket fehlt / API-Fehler. Programm laeuft immer weiter

Datenschutz: nur Vornamen werden uebergeben, keine sensiblen Daten.

7. main.py – Die API (alle Endpunkte)

Verbindet alles. Beim Start: .env laden, Tabellen anlegen, Demo-Daten einspielen. / liefert index.html, /static die Frontend-Dateien. Rund 40 Endpunkte nach gleichem Muster (Beispiel Mitarbeiter):

Methode + Pfad	Funktion
GET .../employees	Liste aller Mitarbeiter
POST .../employees	Neuen anlegen
GET /api/employees/{id}	Einen abrufen
PUT /api/employees/{id}	Aendern
DELETE /api/employees/{id}	Loeschen

Spezial-Endpunkte:

- **POST .../schedule/generate:** ruft den Scheduler. ValueError (kein Montag) wird als HTTP 400 zurueckgegeben.
- **GET/DELETE .../schedule:** Plan einer Woche abrufen/loeschen.
- **GET .../schedule/export/csv:** baut im Speicher eine CSV (Semikolon-getrennt, utf-8-sig fuer korrekte Umlaute in Excel) als Download.
- **POST .../gemini-analysis:** sammelt den Plan, berechnet MA-Stunden, anonymisiert (nur Vorname) und schickt die Zusammenfassung an Gemini.

8. seed.py – Demo-Daten

- Bricht ab, wenn schon ein Betrieb existiert – wird nicht doppelt angelegt.
- Legt 'Demo Bistro' an: Oeffnungszeiten (Mo–Sa 9–23, So zu), Bereiche, Rollen.
- 5 Mitarbeiter: Ali, Sara (Senior), Max (Junior, 2 Rollen), Lena, Tom.
- Anforderungen fuer Mo–Sa (Mittag + Abend).
- Verfuegbarkeiten (z.B. Lena nur vormittags, Max montags nur abends).
- Krankmeldung: Sara ist Donnerstag krank → Kueche wird sichtbar unterbesetzt.
- Alle Daten beziehen sich auf die aktuelle Woche → sofort planbar.

9. Frontend (index.html, styles.css, app.js)

index.html: Single-Page-App – alle Seiten sind vorhanden, nur eine ist sichtbar (active).

Sidebar-Navigation, Betriebsauswahl, Modals fuer Formulare.

styles.css: modernes Design ueber CSS-Variablen (--primary, --danger). Sidebar, Karten, Tabellen, Badges, Schicht-Karten (rot bei Unterbesetzung), Toasts, Modals.

app.js – die Logik:

- **state:** merkt sich Betrieb, Listen, aktuelle Woche.
- **api(method, path, body):** zentrale fetch-Funktion mit lesbarer Fehlerbehandlung.
- **navigate(page):** zeigt Seite und laedt deren Daten bei Bedarf (lazy).
- Pro Bereich ein Set aus load + render + save + delete.
- **renderSchedule:** baut die 7-Spalten-Wochenansicht, filtert nach MA/Bereich, markiert heute, zeigt die Warnungen-Box.
- **generateSchedule / deleteSchedule / exportCSV:** bedienen die Buttons.
- **init():** startet alles; bei Backend-Fehler erscheint ein Toast.

Datenfluss am Beispiel 'Plan erstellen'

1. Klick auf 'Plan erstellen' → generateSchedule() in app.js
2. api('POST', '../schedule/generate', {week_start}) → HTTP-Anfrage
3. main.py-Endpunkt → ruft scheduler.generate_schedule
4. scheduler.py laedt Daten, rechnet, schreibt GeneratedShift + ShiftAssignment in die DB
5. Rueckgabe der Zusammenfassung → Toast 'X Schichten, Y Zuweisungen'
6. renderSchedule() zeichnet die Wochenansicht mit roten Markierungen